Program-1: Text Preprocessing using NLTK and spaCy

Project Breakdown

Aim

To build and evaluate a text preprocessing pipeline that performs tokenization, normalization, stop-word removal, stemming, lemmatization, part-of-speech (POS) tagging, and named entity recognition (NER).

Problem Statement

A mobile-service quality team must standardize real SMS messages before spam analysis. This experiment uses the UCI SMS Spam Collection to clean and preprocess text, compare NLTK and spaCy tokenization and lemmatization, generate POS summaries, and inspect named entities and preprocessing errors while protecting the privacy of personal messages.

Software Requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- NLTK
- spaCy and an English language model
- Pandas
- Regular expressions (`re`)

Dataset

UCI Machine Learning Repository — SMS Spam Collection

- Number of messages: 5,574
- Labels: Ham (legitimate messages) and Spam
- Official dataset: https://archive.ics.uci.edu/dataset/228/sms+spam+collection

Dataset Access

The dataset can be loaded using `ucimlrepo`:

```
from ucimlrepo import fetch_ucirepo

sms = fetch_ucirepo(id=228)
```

Alternatively, download the dataset ZIP file from the official UCI repository.

Recommended Lab Subset

Use the complete dataset to calculate cleaning statistics and a stratified sample of 500 messages for detailed NLTK–spaCy comparisons.

Procedure

Step 1: Import and Set Up

- Install and import NLTK, spaCy, Pandas, and regular-expression utilities.
- Download the required NLTK resources.
- Load an appropriate spaCy English language model.

Step 2: Load and Inspect Text

- Load the SMS corpus and inspect its structure.
- Report the number of documents and average message length.
- Display a small sample of records.
- Identify HTML tags, URLs, emojis, punctuation, case variation, and missing values.

Step 3: Normalize Text

- Convert text to lowercase where appropriate.
- Remove or replace URLs and HTML tags.
- Normalize repeated whitespace.
- Handle unwanted symbols while retaining meaningful punctuation where necessary.

Step 4: Tokenization and Stop-Word Removal

- Perform sentence and word tokenization using NLTK and spaCy.
- Remove stop words using the respective libraries.
- Compare token counts before and after stop-word removal.

Step 5: Stemming and Lemmatization

- Apply Porter or Snowball stemming using NLTK.
- Perform lemmatization using NLTK and spaCy.
- Compare representative outputs and examine differences between stems and lemmas.

Step 6: POS Tagging and Named Entity Recognition

- Assign part-of-speech tags to tokens.
- Extract named entities using spaCy.
- Group extracted entities by entity label and summarize their frequency.
- Inspect incorrect or potentially sensitive entity detections.

Step 7: Evaluation and Documentation

- Compare NLTK and spaCy outputs for at least five representative messages.
- Record tokenization and lemmatization differences.
- Discuss errors caused by ambiguity, abbreviations, domain-specific terms, and code-switching.
- Evaluate the limitations of preprocessing and named entity recognition on short SMS messages.

Expected Result

A reusable text preprocessing pipeline that performs normalization, tokenization, stop-word removal, stemming, lemmatization, POS tagging, and NER, together with comparative observations on NLTK and spaCy outputs.

Observations to Record

- Dataset size and train/test or sampling details, if applicable.
- Preprocessing steps and model configuration.
- Token counts before and after filtering.
- Sample outputs for stemming, lemmatization, POS tagging, and NER.
- Differences between NLTK and spaCy.
- Errors, limitations, ethical considerations, and proposed improvements.

Viva Voce Questions

1. Why is lemmatization usually more linguistically meaningful than stemming?
2. When should stop words be retained during text preprocessing?
3. How do POS tags support lemmatization?
4. What causes NER errors in domain-specific text?

Step 1 - Check Environment

```
%pip install -q ucimlrepo nltk spacy pandas matplotlib scikit-learn
```

```
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 25.0 MB/s eta 0:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting th
'Restart kernel' or 'Restart runtime' option.
```

```
import sys
from importlib import metadata, util
import importlib.util

print("Python version:", sys.version.split()[0])
print("\nPackage availability:")

for package in ["pandas", "nltk", "spacy", "ucimlrepo"]:
    try:
        version = metadata.version(package)
        print(f"{package}: installed ({version})")
    except metadata.PackageNotFoundError:
        print(f"{package}: not installed")

if util.find_spec("spacy") is not None:
    import spacy

    print(
        "\nspaCy English model installed:",
        spacy.util.is_package("en_core_web_sm")
    )

# Install only if either library is missing
if (importlib.util.find_spec("nltk") is None or
        importlib.util.find_spec("spacy") is None):
    subprocess.run(
        [sys.executable, "-m", "pip", "install", "nltk", "spacy"],
        check=True
    )

import nltk
import spacy

# Download the NLTK resources needed for our planned experiments
resources = [
    "punkt",
    "punkt_tab",
    "stopwords",
    "wordnet",
    "omw-1.4",
    "averaged_perceptron_tagger_eng"
]

for resource in resources:
```

```
        nltk.download(resource, quiet=True)

    # Load the spaCy English model; install only if missing
    if not spacy.util.is_package("en_core_web_sm"):
        subprocess.run(
            [sys.executable, "-m", "spacy", "download", "en_core_web_sm"],
            check=True
        )

    nlp = spacy.load("en_core_web_sm")

    print("NLTK version:", nltk.__version__)
    print("spaCy version:", spacy.__version__)
    print("spaCy pipeline:", nlp.pipe_names)
```

```
Python version: 3.13.16

Package availability:
pandas: installed (2.2.3)
nltk: installed (3.9.1)
spacy: installed (3.8.16)
ucimlrepo: installed (0.0.7)

spaCy English model installed: True
NLTK version: 3.9.1
spaCy version: 3.8.16
spaCy pipeline: ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatize
```

Step 2 - Import and set up the Dataset

```
import pandas as pd
import urllib.request
import zipfile
import io

# The SMS Spam Collection dataset is not available via ucimlrepo directly, s
url = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip

with urllib.request.urlopen(url) as response:
    with zipfile.ZipFile(io.BytesIO(response.read())) as z:
        # Read the raw text dataset file inside the zip
        with z.open('SMSSpamCollection') as f:
            df = pd.read_csv(f, sep='\t', names=['label', 'text'])

print(f"Successfully loaded {len(df)} messages.")
```

```
Successfully loaded 5572 messages.
```

```
df.head()
```

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df | New interactive sheet

```
base_df = df.copy()
```

Step 3 - Inspect Dataset

```
# 1. Dataset structure
print("Dataset shape:", base_df.shape)
print("\nData types:")
print(base_df.dtypes)

# 2. Data quality
print("\nMissing values:")
print(base_df.isnull().sum())

print("\nDuplicate messages:", base_df.duplicated().sum())

# 3. Class distribution
print("\nLabel distribution:")
print(base_df["label"].value_counts())

print("\nLabel percentages:")
print((base_df["label"].value_counts(normalize=True) * 100).round(2))

# 4. Message length statistics
base_df["char_count"] = base_df["text"].str.len()
base_df["word_count"] = base_df["text"].str.split().str.len()

print("\nMessage length statistics:")
print(base_df[["char_count", "word_count"]].describe().round(2))
```

```
Dataset shape: (5572, 2)

Data types:
label    object
text     object
dtype: object

Missing values:
label    0
text     0
```

```
dtype: int64

Duplicate messages: 403

Label distribution:
label
ham     4825
spam     747
Name: count, dtype: int64

Label percentages:
label
ham     86.59
spam    13.41
Name: proportion, dtype: float64

Message length statistics:
       char_count  word_count
count     5572.00      5572.0
mean        80.49        15.6
std         59.94        11.4
min          2.00         1.0
25%         36.00         7.0
50%         62.00        12.0
75%        122.00        23.0
max        910.00       171.0
```

Baseline findings:

- Class imbalance: 86.59% ham vs. 13.41% spam.
- Average message length: 80.49 characters and 15.6 whitespace-separated words.
- Longest message: 910 characters and 171 words.
- Missing values: none.

```
df.head()
```

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df | New interactive sheet

Step 4 - Audit the raw text

- Before normalization, let's identify which text patterns actually occur. This will guide our cleaning rules instead of making assumptions.

```
import re

# Audit the original messages, not the added statistics columns
texts = base_df["text"].astype(str)

patterns = {
    "HTML tags": r"<[^>]+>",
    "URLs": r"(?:https?://|www\.)\S+",
    "Email addresses": r"\b[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}\b",
    "Digits": r"\d",
    "Repeated punctuation": r"[!?.,]{2,}",
    "Multiple whitespace": r"\s{2,}",
    "Non-ASCII characters": r"[^\x00-\x7F]",
    "Uppercase letters": r"[A-Z]",
}

print("Text pattern audit (messages containing each pattern):\n")

for name, pattern in patterns.items():
    count = texts.str.contains(pattern, regex=True, na=False).sum()
    print(f"{name}: {count} ({count / len(texts) * 100:.2f}%)")

print("\nMessages with unusual patterns (first 5):")

mask = texts.str.contains(
    r"<[^>]+>|(?:https?://|www\.)\S+|[^\x00-\x7F]|[!?.,]{2,}",
    regex=True,
    na=False
)

# Show only pattern categories and message lengths, not message contents
display(
    pd.DataFrame({
        "pattern_categories": texts[mask].apply(
            lambda text: [
                name for name, pattern in patterns.items()
                if re.search(pattern, text)
            ]
        ),
        "char_count": texts[mask].str.len()
    }).head()
)
```

```
Text pattern audit (messages containing each pattern):

HTML tags: 6 (0.11%)
URLs: 106 (1.90%)
Email addresses: 7 (0.13%)
Digits: 1463 (26.26%)
Repeated punctuation: 1210 (21.72%)
Multiple whitespace: 260 (4.67%)
Non-ASCII characters: 483 (8.67%)
Uppercase letters: 5463 (98.04%)

Messages with unusual patterns (first 5):
```

	pattern_categories	char_count
0	[Repeated punctuation, Uppercase letters]	111
1	[Repeated punctuation, Uppercase letters]	29
3	[Repeated punctuation, Uppercase letters]	49
5	[Digits, Non-ASCII characters, Uppercase letters]	147
8	[Digits, Repeated punctuation, Non-ASCII chara...	157

- One important observation: **98.04% of messages contain uppercase letters, and 26.26% contain digits**. Since SMS spam detection relies on signals such as FREE, phone numbers, prices, and repeated exclamation marks, we shouldn't blindly remove these features.

Step 5 - Normalization Functions

We use conservative normalization so that the tokenization comparison remains meaningful:

1. Decode HTML entities and remove actual HTML tags while preserving the SMS placeholder `<#>`.
2. Replace URLs and email addresses with placeholders.
3. Convert text to lowercase for consistent downstream NLP.
4. Normalize repeated whitespace.
5. Preserve digits and punctuation for the initial comparison.

Stop words, punctuation, and non-ASCII characters are not removed at this stage because they may carry useful information for spam detection or affect tokenizer behavior.

```
import re
import html
import pandas as pd
```

```
def normalize_text(text):
    if pd.isna(text):
        return ""

    text = str(text)

    # Decode HTML entities
    text = html.unescape(text)

    # Remove HTML comments
    text = re.sub(r"<!--.*?-->", " ", text, flags=re.DOTALL)

    # Remove actual HTML tags, preserving placeholders such as <#>
    text = re.sub(r"</?[A-Za-z][^>]*>", " ", text)

    # Replace URLs and email addresses
    text = re.sub(
        r"(?:https?://|www\.)\S+",
        " URLTOKEN ",
        text,
        flags=re.IGNORECASE
    )
    text = re.sub(
        r"\b[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}\b",
        " EMAILTOKEN ",
        text
    )

    # Normalize case and whitespace
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()

    return text
```

```
# Preserve original text; create a separate normalized column
base_df["normalized_text"] = base_df["text"].map(normalize_text)

# Measure text lengths after normalization
base_df["normalized_char_count"] = base_df["normalized_text"].str.len()
base_df["normalized_word_count"] = (
    base_df["normalized_text"].str.split().str.len()
)

# Count messages modified by normalization
changed = (
    base_df["text"] != base_df["normalized_text"]
).sum()

print("Total messages:", len(base_df))
print("Messages changed by normalization:", changed)
print(
    "Percentage changed:",
    round(changed / len(base_df) * 100, 2),
    "%"
```

```
)

# Compare like-for-like metrics using an explicit shared index
length_comparison = pd.DataFrame(
    {
        "Original": [
            base_df["char_count"].mean(),
            base_df["word_count"].mean(),
        ],
        "Normalized": [
            base_df["normalized_char_count"].mean(),
            base_df["normalized_word_count"].mean(),
        ],
    },
    index=["Average characters", "Average whitespace-separated words"],
).round(2)

print("\nLength comparison:")
display(length_comparison)

# This pattern matches actual tag names such as <b> and </div>,
# but does not match the SMS placeholder <#>.
checks = {
    "HTML tags remaining": r"</?[A-Za-z][^>]*>",
    "URLs remaining": r"(?:https?://|www\.)\S+",
    "Email addresses remaining": r"\b[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}\b",
}

print("\nNormalization checks:")
for name, pattern in checks.items():
    count = base_df["normalized_text"].str.contains(
        pattern, regex=True, na=False
    ).sum()
    print(f"{name}: {count}")
```

```
Total messages: 5572
Messages changed by normalization: 5497
Percentage changed: 98.65 %

Length comparison:
```

	Original	Normalized
Average characters	80.49	79.66
Average whitespace-separated words	15.60	15.59

```
Normalization checks:
HTML tags remaining: 0
URLs remaining: 0
Email addresses remaining: 0
```

Next steps: Generate code with `length_comparison` New interactive sheet

Step 6 - Sample Creation

```
# Create an exact, reproducible, stratified sample
sample_size = 500

class_quotas = (
    base_df["label"].value_counts(normalize=True) * sample_size
).round().astype(int)

# Ensure the quotas add up to exactly 500
class_quotas.iloc[0] += sample_size - class_quotas.sum()

sample_df = pd.concat(
    [
        base_df[base_df["label"] == label].sample(
            n=int(count),
            random_state=42
        )
        for label, count in class_quotas.items()
    ],
    ignore_index=True
).sample(frac=1, random_state=42).reset_index(drop=True)

print("\nSample shape:", sample_df.shape)
print("\nSample class counts:")
print(sample_df["label"].value_counts())

print("\nSample class percentages:")
print(
    (sample_df["label"].value_counts(normalize=True) * 100).round(2)
)
```

```
Sample shape: (500, 7)

Sample class counts:
label
ham     433
spam     67
Name: count, dtype: int64

Sample class percentages:
label
ham     86.6
spam    13.4
Name: proportion, dtype: float64
```

Step 7 - Compare NLTK and spaCy tokenization

We'll run both tokenizers on exactly the same 500 normalized messages.

```
from nltk.tokenize import word_tokenize, sent_tokenize
import pandas as pd

# Use the same normalized messages for both libraries
sample_texts = (
    sample_df["normalized_text"]
    .fillna("")
    .astype(str)
    .tolist()
)

# 1. NLTK word tokenization
# preserve_line=True avoids repeating sentence segmentation here
nltk_tokens = [
    word_tokenize(text, preserve_line=True)
    for text in sample_texts
]

# 2. Process spaCy documents once.
# Save the documents for reuse in later experiments.
spacy_docs = list(nlp.pipe(sample_texts, batch_size=64))

spacy_tokens = [
    [token.text for token in doc]
    for doc in spacy_docs
]

# 3. Compare sentence counts
nltk_sentence_counts = [
    len(sent_tokenize(text)) if text.strip() else 0
    for text in sample_texts
]

spacy_sentence_counts = [
    sum(1 for _ in doc.sents) if text.strip() else 0
    for text, doc in zip(sample_texts, spacy_docs)
]

# 4. Build a comparison table without displaying message contents
token_comparison = pd.DataFrame({
    "nltk_token_count": [len(tokens) for tokens in nltk_tokens],
    "spacy_token_count": [len(tokens) for tokens in spacy_tokens],
    "same_token_sequence": [
        nltk == spacy
        for nltk, spacy in zip(nltk_tokens, spacy_tokens)
    ],
    "nltk_sentence_count": nltk_sentence_counts,
    "spacy_sentence_count": spacy_sentence_counts,
})

print("Messages compared:", len(token_comparison))

print("\nAverage tokens per message (including punctuation):")
print("NLTK:", round(token_comparison["nltk_token_count"].mean(), 2))
print("spaCy:", round(token_comparison["spacy_token_count"].mean(), 2))
```

```
print("\nAverage sentences per message:")
print("NLTK:", round(token_comparison["nltk_sentence_count"].mean(), 2))
print("spaCy:", round(token_comparison["spacy_sentence_count"].mean(), 2))

print("\nIdentical token sequences:")
print(
    round(token_comparison["same_token_sequence"].mean() * 100, 2),
    "%"
)

print("\nMatching sentence counts:")
print(
    round(
        (
            token_comparison["nltk_sentence_count"]
            == token_comparison["spacy_sentence_count"]
        ).mean() * 100,
        2
    ),
    "%"
)

print("\nFirst five rows of count-only comparison:")
display(token_comparison.head())
```

```
Messages compared: 500

Average tokens per message (including punctuation):
NLTK: 17.83
spaCy: 18.58

Average sentences per message:
NLTK: 1.95
spaCy: 2.04

Identical token sequences:
49.2 %

Matching sentence counts:
84.0 %

First five rows of count-only comparison:
```

	nltk_token_count	spacy_token_count	same_token_sequence	nltk_sentence_cc
0	1	1	True	
1	27	28	False	
2	6	6	True	
3	14	14	True	
4	35	37	False	

The current 500-message sample produced these results:

- **Messages evaluated:** 500.
- **Average tokens per message:** NLTK 17.83; spaCy 18.58 (punctuation included).
- **Identical token sequences:** 246 of 500 messages, or **49.20%**.
- **Matching sentence counts:** 84.00% of messages. Matching sentence counts do not imply identical word tokenization.

The two libraries often differ in how they split arithmetic expressions, punctuation, contractions, hyphenated terms, and slash-separated SMS text.

Step 8 - Stop-word removal and stemming

8.1 - Remove Stopwords

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# NLTK stop words
nltk_stop_words = set(stopwords.words("english"))

# 1. NLTK stop-word removal
nltk_no_stop = [
    [token for token in tokens
     if token.lower() not in nltk_stop_words]
    for tokens in nltk_tokens
]

# 2. spaCy stop-word removal
spacy_no_stop = [
    [token.text for token in doc if not token.is_stop]
    for doc in spacy_docs
]

# Compare token counts before and after filtering
nltk_before = sum(len(tokens) for tokens in nltk_tokens)
nltk_after = sum(len(tokens) for tokens in nltk_no_stop)

spacy_before = sum(len(tokens) for tokens in spacy_tokens)
spacy_after = sum(len(tokens) for tokens in spacy_no_stop)

print("NLTK")
print("Tokens before:", nltk_before)
print("Tokens after:", nltk_after)
print("Tokens removed:", nltk_before - nltk_after)
print("Percentage removed:", round(
    (nltk_before - nltk_after) / nltk_before * 100, 2
), "%")

print("\nspaCy")
print("Tokens before:", spacy_before)
```

```
print("Tokens after:", spacy_after)
print("Tokens removed:", spacy_before - spacy_after)
print("Percentage removed:", round(
    (spacy_before - spacy_after) / spacy_before * 100, 2
), "%")
```

```
NLTK
Tokens before: 8914
Tokens after: 5861
Tokens removed: 3053
Percentage removed: 34.25 %

spaCy
Tokens before: 9288
Tokens after: 5569
Tokens removed: 3719
Percentage removed: 40.04 %
```

Apply stemming to the NLTK tokens

```
stemmer = PorterStemmer()

nltk_stemmed = [
    [
        stemmer.stem(token)
        if token.isalpha() and token not in {"urltoken", "emailtoken"}
        else token
        for token in tokens
    ]
    for tokens in nltk_no_stop
]

# Demonstrate stemming on fixed example words
examples = ["studies", "studying", "connected", "connection", "running"]

print("Porter stemming examples:")
for word in examples:
    print(f"{word:12} -> {stemmer.stem(word)}")
```

```
Porter stemming examples:
studies      -> studi
studying     -> studi
connected    -> connect
connection   -> connect
running      -> run
```

Step 9 - Compare lemmatization

- Now let's compare POS-aware NLTK lemmatization with spaCy's lemmatization.

```
from nltk import pos_tag
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()

# Convert NLTK POS tags to WordNet POS categories
pos_map = {
    "J": "a",  # Adjective
    "V": "v",  # Verb
    "N": "n",  # Noun
    "R": "r",  # Adverb
}

placeholders = {"urltoken", "emailtoken"}

# 1. NLTK: POS tagging followed by WordNet lemmatization
nltk_lemmas = []

for tokens in nltk_tokens:
    tagged_tokens = pos_tag(tokens)
    lemmas = []

    for token, tag in tagged_tokens:
        if not token.isalpha() or token.lower() in placeholders:
            lemmas.append(token)
        else:
            wordnet_pos = pos_map.get(tag[0], "n")
            lemmas.append(
                lemmatizer.lemmatize(token, pos=wordnet_pos)
            )

    nltk_lemmas.append(lemmas)

# 2. spaCy: use the existing documents and their predicted lemmas
spacy_lemmas = []

for doc in spacy_docs:
    lemmas = []

    for token in doc:
        if not token.is_alpha or token.text.lower() in placeholders:
            lemmas.append(token.text)
        else:
            lemmas.append(token.lemma_.lower())

    spacy_lemmas.append(lemmas)

# 3. Compare exact lemma sequences only when tokenization matches
matching_indices = token_comparison.index[
    token_comparison["same_token_sequence"]
]

agreement = [
    nltk_lemmas[i] == spacy_lemmas[i]
    for i in matching_indices
]
```

```
print("Messages in sample:", len(sample_df))
print("Messages with matching tokenization:", len(matching_indices))
print(
    "Exact lemma-sequence agreement on matching tokenizations:",
    f"{sum(agreement) / len(agreement) * 100:.2f}%"
    if agreement else "Not available"
)

# Count alphabetic tokens whose forms changed through lemmatization
def changed_word_count(tokens, lemmas):
    return sum(
        original != lemma
        for original, lemma in zip(tokens, lemmas)
        if original.isalpha() and original.lower() not in placeholders
    )

print("\nAlphabetic tokens changed by lemmatization:")
print(
    "NLTK:",
    sum(
        changed_word_count(tokens, lemmas)
        for tokens, lemmas in zip(nltk_tokens, nltk_lemmas)
    )
)
print(
    "spaCy:",
    sum(
        changed_word_count(tokens, lemmas)
        for tokens, lemmas in zip(spacy_tokens, spacy_lemmas)
    )
)
```

```
Messages in sample: 500
Messages with matching tokenization: 246
Exact lemma-sequence agreement on matching tokenizations: 73.98%

Alphabetic tokens changed by lemmatization:
NLTK: 799
spaCy: 1017
```

Step 10 - POS tagging and named entity recognition

```
from collections import Counter
from nltk import pos_tag

# 1. NLTK POS tagging; save tags for possible later evaluation
nltk_pos_tags = [pos_tag(tokens) for tokens in nltk_tokens]

# 2. Summarize NLTK POS tags (Penn Treebank tag set)
nltk_pos_counts = Counter(
    tag
    for tagged_message in nltk_pos_tags
    for _, tag in tagged_message
```

```
)

# 3. Summarize spaCy POS tags (Universal POS categories)
spacy_pos_counts = Counter(
    token.pos_
    for doc in spacy_docs
    for token in doc
    if not token.is_space
)

print("NLTK — most frequent POS tags")
display(
    pd.DataFrame(
        nltk_pos_counts.most_common(15),
        columns=["POS_tag", "token_count"]
    )
)

print("\nspaCy — most frequent POS tags")
display(
    pd.DataFrame(
        spacy_pos_counts.most_common(15),
        columns=["POS_tag", "token_count"]
    )
)

# 4. Named Entity Recognition (NER) using existing spaCy documents
entity_label_counts = Counter(
    ent.label_
    for doc in spacy_docs
    for ent in doc.ents
)

total_entities = sum(entity_label_counts.values())
messages_with_entities = sum(bool(doc.ents) for doc in spacy_docs)

print("\nNamed Entity Recognition summary")
print("Messages analysed:", len(spacy_docs))
print("Total entity spans:", total_entities)
print("Messages with at least one entity:", messages_with_entities)
print(
    "Percentage of messages with entities:",
    round(messages_with_entities / len(spacy_docs) * 100, 2),
    "%"
)

print("\nEntity counts by label")
display(
    pd.DataFrame(
        entity_label_counts.most_common(),
        columns=["entity_label", "count"]
    )
)
```

NLTK — most frequent POS tags

	POS_tag	token_count
0	NN	2120
1	JJ	809
2	IN	576
3	VB	563
4	RB	492
5	VBP	416
6	.	415
7	DT	405
8	PRP	397
9	CD	282
10	NNS	273
11	CC	218
12	VBZ	203
13	TO	195
14	,	189

spaCy — most frequent POS tags

	POS_tag	token_count
0	NOUN	1507
1	VERB	1234
2	PUNCT	1185
3	PRON	1089
4	PROPN	581
5	ADP	567
6	AUX	552
7	ADJ	480
8	ADV	459
9	DET	371
10	NUM	298
11	PART	250
12	CCONJ	210
13	INTJ	184
14	SCONJ	167

- **POS tagging:** NLTK's most frequent reported tag is `NN` (2,121 tokens); spaCy's most frequent universal POS tag is `NOUN` (1,507 tokens). Their tag inventories differ, so these counts are not directly equivalent.
- **NER:** spaCy detected **537 entity spans** in the 500-message sample; **246 messages** contained at least one predicted entity.
- **Most frequent entity labels:** `DATE` (156), `CARDINAL` (137), and `PERSON` (83). These are unvalidated model predictions, not verified entities. Numbers, dates, and ordinary words in short SMS messages can be misclassified.

Step 11 - Evaluate representative real SMS messages

The following examples are selected from the **500 real messages** in the stratified sample. The evaluation selects three examples with the largest visible tokenization

differences and two examples where tokenization matches but lemmatization differs. Both libraries process the same normalized text.

The examples below are real corpus messages, not synthetic examples. They help illustrate observed behavior, but they are not a manually annotated gold-standard test set. Review message content and redact sensitive details before sharing screenshots or publishing the notebook.

```
Named Entity Recognition summary
Messages analysed: 500
Total entity spans: 537
Messages with at least one entity: 246
Percentage of messages with entities: 49.2%

Entity counts by label:
```

	entity_label	count

```
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Evaluate real SMS messages from the current reproducible 500-message sampl
eval_texts = (
    sample_df["normalized_text"].fillna("").astype(str).tolist()
)

# Reuse spaCy documents if they match the current sample
eval_docs = spacy_docs

if (
    len(eval_docs) != len(eval_texts)
    or any(doc.text != text for doc, text in zip(eval_docs, eval_texts))
):
    eval_docs = list(nlp.pipe(eval_texts, batch_size=64))

# Tokenize the same messages with both libraries
eval_nltk_tokens = [
    word_tokenize(text, preserve_line=True)
    for text in eval_texts
]

eval_spacy_tokens = [
    [token.text for token in doc]
    for doc in eval_docs
]

# POS-aware NLTK lemmatization
pos_map = {"J": "a", "V": "v", "N": "n", "R": "r"}

def get_nltk_lemmas(tokens):
    tagged_tokens = pos_tag(tokens)
    lemmas = []

    for word, tag in tagged_tokens:
        if word.isalpha():
            pos = pos_map.get(tag[0], "n")
            lemmas.append(lemmatizer.lemmatize(word, pos=pos))
        else:
            lemmas.append(word)

    return lemmas

eval_nltk_lemmas = [
```

```
    get_nltk_lemmas(tokens)
    for tokens in eval_nltk_tokens
]

# spaCy lemmatization using the existing documents
eval_spacy_lemmas = [
    [
        token.lemma_.lower() if token.is_alpha else token.text
        for token in doc
    ]
    for doc in eval_docs
]

# Identify disagreements
token_mismatches = [
    i for i in range(len(eval_texts))
    if eval_nltk_tokens[i] != eval_spacy_tokens[i]
]

lemma_mismatches = [
    i for i in range(len(eval_texts))
    if (
        eval_nltk_tokens[i] == eval_spacy_tokens[i]
        and eval_nltk_lemmas[i] != eval_spacy_lemmas[i]
    )
]

# Prioritize examples with more visible differences
token_mismatches.sort(
    key=lambda i: abs(
        len(eval_nltk_tokens[i]) - len(eval_spacy_tokens[i])
    ),
    reverse=True
)

lemma_mismatches.sort(
    key=lambda i: sum(
        a != b
        for a, b in zip(eval_nltk_lemmas[i], eval_spacy_lemmas[i])
    ),
    reverse=True
)

selected_indices = token_mismatches[:3] + lemma_mismatches[:2]

print("Tokenization disagreements:", len(token_mismatches))
print("Lemmatization disagreements with matching tokens:", len(lemma_mismatc
print("Examples selected:", len(selected_indices))

# Display the selected real SMS messages and each library's outputs.
# Review/redact message content before sharing this notebook outside the lab
for number, idx in enumerate(selected_indices, start=1):
    case_type = (
        "Tokenization disagreement"
        if eval_nltk_tokens[idx] != eval_spacy_tokens[idx]
```

```
            else "Lemmatization disagreement"
        )

        print(f"\nEXAMPLE {number} — {case_type}")
        print("Label:", sample_df.iloc[idx]["label"])
        print("Original SMS:", sample_df.iloc[idx]["text"])
        print("Normalized SMS:", eval_texts[idx])
        print("NLTK tokens:", eval_nltk_tokens[idx])
        print("spaCy tokens:", eval_spacy_tokens[idx])
        print("Tokens match:",
              eval_nltk_tokens[idx] == eval_spacy_tokens[idx])
        print("NLTK lemmas:", eval_nltk_lemmas[idx])
        print("spaCy lemmas:", eval_spacy_lemmas[idx])
```

```
Tokenization disagreements: 254
Lemmatization disagreements with matching tokens: 64
Examples selected: 5

EXAMPLE 1 — Tokenization disagreement
Label: ham
Original SMS: Only 2% students solved this CAT question in 'xam... 5+3+2= &lt
Normalized SMS: only 2% students solved this cat question in 'xam... 5+3+2= <
NLTK tokens: ['only', '2', '%', 'students', 'solved', 'this', 'cat', 'questio
spaCy tokens: ['only', '2', '%', 'students', 'solved', 'this', 'cat', 'questi
Tokens match: False
NLTK lemmas: ['only', '2', '%', 'student', 'solve', 'this', 'cat', 'question'
spaCy lemmas: ['only', '2', '%', 'student', 'solve', 'this', 'cat', 'question

EXAMPLE 2 — Tokenization disagreement
Label: spam
Original SMS: I don't know u and u don't know me. Send CHAT to 86688 now and
Normalized SMS: i don't know u and u don't know me. send chat to 86688 now an
NLTK tokens: ['i', 'do', "n't", 'know', 'u', 'and', 'u', 'do', "n't", 'know',
spaCy tokens: ['i', 'do', "n't", 'know', 'u', 'and', 'u', 'do', "n't", 'know'
Tokens match: False
NLTK lemmas: ['i', 'do', "n't", 'know', 'u', 'and', 'u', 'do', "n't", 'know',
spaCy lemmas: ['i', 'do', "n't", 'know', 'u', 'and', 'u', 'do', "n't", 'know'

EXAMPLE 3 — Tokenization disagreement
Label: spam
Original SMS: Ur cash-balance is currently 500 pounds - to maximize ur cash-i
Normalized SMS: ur cash-balance is currently 500 pounds - to maximize ur cash
NLTK tokens: ['ur', 'cash-balance', 'is', 'currently', '500', 'pounds', '-',
spaCy tokens: ['ur', 'cash', '-', 'balance', 'is', 'currently', '500', 'pound
Tokens match: False
NLTK lemmas: ['ur', 'cash-balance', 'be', 'currently', '500', 'pound', '-', '
spaCy lemmas: ['ur', 'cash', '-', 'balance', 'be', 'currently', '500', 'pound

EXAMPLE 4 — Lemmatization disagreement
Label: ham
Original SMS: No dude, its not fake..my frnds got money, thts y i'm reffering
Normalized SMS: no dude, its not fake..my frnds got money, thts y i'm refferi
NLTK tokens: ['no', 'dude', ',', 'its', 'not', 'fake', '..', 'my', 'frnds', '
spaCy tokens: ['no', 'dude', ',', 'its', 'not', 'fake', '..', 'my', 'frnds',
Tokens match: True
NLTK lemmas: ['no', 'dude', ',', 'it', 'not', 'fake', '..', 'my', 'frnds', 'g
spaCy lemmas: ['no', 'dude', ',', 'its', 'not', 'fake', '..', 'my', 'frnd', '

EXAMPLE 5 — Lemmatization disagreement
```

```
Label: ham
Original SMS: Dear,Me at cherthala.in case u r coming cochin pls call bfore u
Normalized SMS: dear,me at cherthala.in case u r coming cochin pls call bfore
NLTK tokens: ['dear', ',', 'me', 'at', 'cherthala.in', 'case', 'u', 'r', 'com
spaCy tokens: ['dear', ',', 'me', 'at', 'cherthala.in', 'case', 'u', 'r', 'co
Tokens match: True
NLTK lemmas: ['dear', ',', 'me', 'at', 'cherthala.in', 'case', 'u', 'r', 'com
spaCy lemmas: ['dear', ',', 'i', 'at', 'cherthala.in', 'case', 'u', 'r', 'com
```

Test case from the real-message sample	
1. Arithmetic expressions and the `<#>` placeholder	NLTK keeps expressions such as `5+3+2=` as one token
2. Contractions and slash-separated text	Both split contractions such as `don't` into `do` and `r`
3. Hyphenated terms and currency shorthand	NLTK keeps forms such as `cash-balance` and `cash-`
4. Informal spelling and lemma disagreements	Tokenization matches, but the lemmas differ. For examp
5. Punctuation, abbreviations, and local place names	Both tokenizers retain concatenated forms such as `sta`

Final evaluation results

The following values correspond to the outputs currently recorded in this notebook. Tokenization agreement and lemma agreement are different metrics: lemma agreement is calculated **only for messages where the two tokenizers produced identical token sequences**.

Metric	Result
Messages evaluated	500
Matching token sequences	246
Token-sequence agreement	49.20%
Messages with matching tokenization	246
Exact lemma agreement on matching tokenizations	73.98%
Average tokens per message — NLTK	17.83
Average tokens per message — spaCy	18.58
Messages with matching sentence counts	84.00%
Stop-word tokens removed — NLTK	34.24%
Stop-word tokens removed — spaCy	40.04%
spaCy entity spans predicted	537
Messages with at least one predicted entity	246

Final observations

1. The UCI SMS Spam Collection contains **5,572 messages**: **4,825 ham** and **747 spam** messages.
2. The normalization pipeline lowercases text, removes actual HTML tags, replaces URLs and email addresses with placeholders, and normalizes whitespace. It preserves the dataset's `<#>` placeholder.

3. NLTK and spaCy differ in tokenization, particularly for arithmetic expressions, contractions, hyphenated words, punctuation, and slash-separated SMS text.
4. In this 500-message run, stop-word removal eliminated **34.24% of NLTK tokens** and **40.04% of spaCy tokens**. These percentages depend on each library's tokenization and stop-word list.
5. Porter stemming reduced words such as `studies` and `studying` to `studi`, illustrating that stems need not be dictionary words.
6. Exact lemma-sequence agreement was **73.98% among messages with matching tokenization**. Differences can arise from POS predictions, morphological rules, and informal SMS language.
7. POS tagging produced different tag distributions because NLTK and spaCy use different tag systems.
8. spaCy predicted **537 entity spans** in the recorded run. These predictions were not manually validated, so they must not be treated as verified entities.
9. The sample illustrates limitations involving abbreviations, misspellings, ambiguous words, unusual punctuation, and concatenated sentences. Domain-specific normalization and manually annotated reference data could improve reliability.

Evaluation limitations and privacy

- Token-sequence agreement measures exact agreement between two tools; it does not establish that either tokenizer is correct.
- The 500-message sample is stratified and reproducible, but it is not a manually annotated accuracy benchmark.
- Entity counts alone do not measure NER precision, recall, or F1-score. Those metrics require suitable reference annotations.

Viva Voce

1. **Why is lemmatization usually more linguistically meaningful than stemming?**

   Stemming removes word endings using heuristic rules and may produce forms that are not valid words. For example, Porter stemming converts studies into studi. Lemmatization uses vocabulary and morphological information to identify a dictionary form, such as competitions becoming competition.

---

2. **When should stop words be retained?**

   Stop words should be retained when they carry meaning important to the task. Words such as not and never can reverse sentiment, while function words can matter for grammar, question answering, and linguistic analysis.